Final project for Python is here!! Here's the full instructions and the 2 attachments. Is it clear what we should do?

`Worked for 24s`

OH YES, my boy! 💪🔥 The Python final project is here — Survive the Quarter.

I've read the entire 12-page brief, not just the Teams instructions. And yes, the assignment is clear. It's more substantial than our previous exercises, but everything builds on what we've already practiced: loops, functions, lists, dictionaries, CSV files and input validation.

1. What we're building

You're the marketing manager of *Croissant Rocket*, a startup delivering croissants by drone.

Starting budget

€10,000

Reputation

50/100

Game length

10 days

Each day, a random marketing crisis appears. You choose A, B or C, affecting your budget and reputation. If either reaches zero or below, you're fired immediately.

If you survive all ten days, your score determines your title.

Final score	Title
Below 95	Eternal Intern
95–119	Growth Hacker
120–139	CMO Material
140+	Viral Legend

The score is calculated as:

`reputation + budget // 200`

2. What the Python program must do

The brief requires us to build a game that:

1. Asks the player's name and loads the scenarios from your CSV.
2. Shuffles the scenarios and plays up to ten days.
3. Displays the current budget, reputation, situation and three options.
4. Validates the player's choice, accepting uppercase or lowercase letters.
5. Applies the financial and reputational consequences, showing feedback.
6. Stops immediately if either gauge reaches zero, otherwise awards a final title.
7. Identifies the best and worst moves based on reputation changes.
8. Saves the game result to `results.csv` without overwriting previous games.

There are also technical requirements: at least four functions, tuples for choices, a tuple-returning function, a history of moves and the standard Python `csv` module.

3. The two extra requirements

Three original scenarios: We must add scenarios 31, 32 and 33 to your assigned CSV, following its exact structure.
One special feature: We must implement one of these four options.

Feature	Effect	Complexity
A — Streak bonus	Three consecutive reputation gains earn €500	Medium
B — The algorithm	Doubles reputation change on one random day	Medium
C — High score	Compares final budget with previous results	Medium
D — Difficulty	Easy starts with €15,000, hard with €6,000	Low

I'd recommend D — Difficulty, because it's straightforward to implement and test. It also provides a meaningful gameplay choice.

4. What we must submit

survive_the_quarter.py
Working game, introductory answers and explanatory comments
scenarios_A.csv
Original scenarios plus three new ones
results.csv
At least three recorded games
AI log (Word or PDF)
Full AI conversation, including corrections
One particularly important detail: the professor wants the full AI conversation, not merely a summary like our previous `CONVERSATION.md` files. We should preserve the actual exchanges as we work.

5. I've also inspected your actual `scenarios_A.csv`

It contains 30 scenarios, with a comma separator and 14 columns:
`ID, Scenario, OptionA, OptionB, OptionC, BudgetA, BudgetB, BudgetC, ReputationA, ReputationB, ReputationC, FeedbackA, FeedbackB, FeedbackC`
Budget and reputation changes are written as signed integers, such as `-800` and `15`.
We can use Python's `csv.DictReader` to load the file.
That means we have the necessary structure to build the game specifically for Group A.

One thing to confirm before we start

Has your professor explicitly assigned you Group A?
I ask because the brief warns that Groups A, B and C use different CSV structures.
Once you confirm, we'll proceed with Group A and build the project step by step in Codespaces, prioritizing a working game and all four deliverables before the deadline.

Yes let's just use group A. Let's go my man!!

LET'S GOOOOO, MY MAN!! 💪🔥

Python Final Project — Survive the Quarter. Group A it is.

We're going to approach this strategically: get a fully functioning game first, then polish it to satisfy every requirement in the professor's brief.

We'll work in your GitHub Codespace, just like the previous exercises. I'll guide you through the steps and we'll test everything before submitting.

Our plan

1. Set up the project — Create the Python file and organize the Group A CSV.
2. Write the game — Implement the functions, ten-day loop, choices, budget and reputation.
3. Add the required features — Difficulty selection, best/worst moves and final titles.
4. Create three original scenarios — Add scenarios 31–33 to the CSV.
5. Test and submit — Play three games, verify `results.csv`, prepare the AI conversation log and push to GitHub.

Step 1 — Set up your Codespace

Open your `pyexercises` GitHub Codespace.

Inside the `examples` folder:

1. Create a new folder called `final_project`.
2. Inside it, create a Python file called `survive_the_quarter.py`.
3. Upload your existing `scenarios_A.csv` into the same folder.

Your Explorer should look like this:

pyexercises

examples

final_project

survive_the_quarter.py

scenarios_A.csv

Important: Don't modify the CSV yet. We'll add the three original scenarios after the basic game works.

Once you've created the folder and placed both files inside, tell me DONE.

Then we'll write the four introductory answers and start coding. One step at a time, but we'll keep the pace fast.

Done my boy

YESSIRRR! 💪🔥 Time to start building.

Step 2 — Answer the four introductory questions

Open `survive_the_quarter.py` in Codespaces.

At the very top, paste this:

```
# FINAL PROJECT: SURVIVE THE QUARTER# Name: Giuseppe Palermo# Group: A# Special feature chosen: D - Difficulty selection## 1. WHAT GOES IN?# The player's name, difficulty level, and choices (A, B or C).# The program also reads marketing scenarios from scenarios_A.csv.## 2. WHAT HAPPENS TO THE INPUT?# The program loads and shuffles the scenarios, then presents# one situation per day for up to 10 days.# Each choice changes the player's budget and reputation.# The game stops if either reaches zero or after day 10.## 3. WHAT COMES OUT?# The program displays the consequences of each decision,# the remaining budget and reputation, and the final result.# It also saves game results to results.csv.## 4. WHAT DO WE NEED TO CALCULATE?# The updated budget and reputation after each decision,# the final score, and the best and worst reputation changes.# We also need to count the number of days survived.## MY CHOSEN FEATURE:# Difficulty selection: Easy starts with 15000 euros,# while Hard starts with 6000 euros.
```

These answers establish the program's inputs, processing, outputs and calculations.

One important point: This is our initial draft. We'll verify the answers and the difficulty settings against the professor's brief as we implement the game.

Once you've pasted the comments, press Cmd + S.

Tell me DONE, and we'll move straight to Step 3: writing the functions that load your CSV and manage the game's budget and reputation.

We're going to get this running today.

done

Perfect, my boy! 💪🔥

Step 3 — Create the first two functions

Now we'll teach Python how to read your Group A scenarios and update the budget and reputation.

Go to the bottom of `survive_the_quarter.py` and paste this immediately below your introductory comments:

```
import csvimport randomfrom pathlib import Path# Find the scenario file in the same folder as this program.FOLDER = Path(__file__).parentSCENARIO_FILE = FOLDER / "scenarios_A.csv"RESULTS_FILE = FOLDER / "results.csv"# Load the
```

```
marketing scenarios from the CSV file.def load_scenarios(filename):
scenarios = []    with open(filename, "r", encoding="utf-8-sig",
newline="") as file:        reader = csv.DictReader(file)        for row in
reader:            scenarios.append(row)    return scenarios# Calculate how
a decision changes the two gauges.# Return both updated values as a
tuple.def apply_choice(budget, reputation, scenario, choice):
budget_change = int(scenario["Budget" + choice])    reputation_change =
int(scenario["Reputation" + choice])    budget += budget_change
reputation += reputation_change    return budget, reputation
```

What these functions do

`load_scenarios()` reads the CSV and stores each marketing crisis as a dictionary. This means we can access information using column names such as `Scenario`, `OptionA` and `BudgetA`.

`apply_choice()` takes the player's decision and calculates the updated budget and reputation. It returns two values together, satisfying the brief's tuple-return requirement.

For example, if a decision costs €800 and increases reputation by 15:

Budget

€10,000 → €9,200

Reputation

50 → 65

Your turn

Paste the code and press Cmd + S.

Don't run it yet. We've only defined the functions; the actual game loop comes next.

Tell me DONE, and we'll build the ten-day gameplay.

Step 6 — Final score, job title and best/worst decisions

Now we'll make the game evaluate how well the player performed.

Go to the bottom of `survive_the_quarter.py`, immediately after `play_game()`, and paste:

```
# Calculate the player's final score and assign a job title.def
calculate_title(budget, reputation):    score = reputation + budget // 200
if score < 95:        title = "Eternal Intern"    elif score < 120:
title = "Growth Hacker"    elif score < 140:        title = "CMO Material"
else:        title = "Viral Legend"    return score, title# Display the
```

```
final results and the best and worst decisions.def
show_final_results(budget, reputation, history):    print("\n==========
FINAL RESULTS ==========")    print(f"Final budget: {budget} EUR")
print(f"Final reputation: {reputation}/100")    print(f"Days played:
{len(history)}")    if budget <= 0 or reputation <= 0:
print("Outcome: FIRED")    else:        print("Outcome: SURVIVED THE
QUARTER!")    score, title = calculate_title(budget, reputation)
print(f"Final score: {score}")    print(f"Your title: {title}")    if
history:        best_move = max(history, key=lambda move: move[2])
```

What this adds

The professor's scoring formula is:

$$\text{Score}=\text{Reputation}+\left\lfloor\frac{\text{Budget}}{200}\right\rfloor$$

Your final title depends on your score:

Score	Title
Below 95	Eternal Intern
95–119	Growth Hacker
120–139	CMO Material
140+	Viral Legend

The game also identifies the best and worst decisions using the reputation changes stored in `history`.

One detail: We're currently using the raw reputation change from the CSV, as specified by the assignment. If reputation is already near 100, the actual increase may be smaller because of the cap.

Your turn

Paste the code and press Cmd + S.

Don't run it yet. Next we'll add the `results.csv` saving function and connect everything so we can play our first complete game.

Tell me DONE!

done

LET'S GOOOO! 🔥 We're one step away from playing our first complete game.

Step 7 — Save the results and launch the game

Go to the bottom of `survive_the_quarter.py` and paste this code below `show_final_results()`:

```
from datetime import datetime# Save the player's result without deleting
previous games.def save_results(name, budget, reputation, title):
file_exists = RESULTS_FILE.exists()    with open(RESULTS_FILE, "a",
encoding="utf-8", newline="") as file:        writer = csv.writer(file)
# Write the header only when creating a new file.        if not file_exists
or RESULTS_FILE.stat().st_size == 0:            writer.writerow(["name",
"budget", "reputation", "title", "date"])        writer.writerow([
name,            budget,            reputation,            title,
datetime.now().strftime("%Y-%m-%d")        ])# Connect the game, final
results and CSV saving.def main():    name, budget, reputation, history =
play_game()    show_final_results(budget, reputation, history)    score,
title = calculate_title(budget, reputation)    save_results(name, budget,
reputation, title)    print("\nYour result has been saved to
results.csv!")# Start the game when this Python file is executed.
```

This completes the main program: it runs the game, displays the final outcome and appends a result to `results.csv` without overwriting previous games.

Step 8 — Run your first game!

Save with Cmd + S, open the terminal in Codespaces and enter:

```
cd /workspaces/pyexercises/examples/final_project
python survive_the_quarter.py
```

You should see something like:

```
=== SURVIVE THE QUARTER ===
Enter your name:
```

Enter your name, select `Easy` or `Hard`, then start choosing A, B or C.

Play through the game until you survive ten days or get fired.

After finishing, check that `results.csv` appears in your `final_project` folder.

One important thing: we still need to add scenarios 31–33, refine the introductory comments to match all four questions in the brief and verify the game against the assignment requirements.

Tell me what happens when you run it. If it works, we'll move straight to the three original scenarios. If there's an error, send me the terminal output and we'll fix it.

My boy, I'm your Chief Strategy Officer now. 😎🚀

For Day 1, choose C:

“Film your bakers at 4 a.m.: quality over speed.”

It's a sensible marketing response to a competitor's faster delivery: emphasize quality rather than destroy your margins with a price war.

Type C and press Enter.

Then send me the Day 2 scenario with its three options. Since the game shuffles scenarios randomly, I'll guide you through each decision as it appears.

Our mission: survive all 10 days with the highest possible score. 🏆

DAY 2 — CHOOSE A! 🎡🔥

“Share it and credit the artist.”

This is a great opportunity to build community goodwill without spending heavily. No reason to threaten a fan who's giving us free publicity!

Our current position:

- Budget: €14,200
- Reputation: 65/100

Type A, press Enter and show me Day 3.

We're going for Viral Legend, my boy! 🏆

DAY 3 — CHOOSE B! 🔥

“Upgrade to silent propellers.”

This solves the underlying noise problem rather than temporarily appeasing residents with free croissants. It's the strongest long-term business decision.

Our current position: €14,200 budget and 75/100 reputation.

Type B, press Enter and show me Day 4.

We're crushing it, my boy! 🎡🚀

DAY 8 — CHOOSE B! 🥐🎡

“Notify everyone and fix your security.”

A breach exposing 5,000 customer addresses is serious. We need to inform affected customers and fix the vulnerability immediately.

Option C adds free croissants, but that's an unnecessary expense when our reputation is already at 100.

Current position: €11,900 | 100/100 reputation.

Type B, press Enter and show me Day 9!

Just two days left, my boy! 🚀

DAY 9 — CHOOSE B! 🎯🚀

“Invite him to visit the bakery.”

Rather than simply reposting a LinkedIn compliment, we can build a genuine relationship and potentially earn more publicity.

We're sitting comfortably at €10,400 and 100/100 reputation.

Type B, press Enter and show me Day 10 — THE FINAL ROUND! 🏆

DAY 10 — CHOOSE B! 🐱🎯🏆

“Send free croissants, no fee.”

Two million followers is a massive publicity opportunity, and sending a few croissants is much cheaper than paying €2,000 for a sponsorship.

Our current position is excellent:

- Budget: €10,200
- Reputation: 100/100
- Days survived: 9/10

Type B and press Enter!

If we finish with €10,200 and 100 reputation, our score would be 151 — Viral Legend! 🔥

After the final results appear, send me a screenshot. We'll verify that `results.csv` was created correctly, then move on to our three original scenarios and the remaining submission requirements.

WE'RE ABOUT TO SURVIVE THE QUARTER, MY BOY! 🚀

My boy, the command worked! 💪

In your screenshot, the terminal returns to the `$` prompt without an error. That means `save_results()` ran successfully.

The `NameError` you see above is from the previous attempt, before we fixed the import.

One important distinction: the terminal doesn't display the saved date. It's written directly into `results.csv`.

Let's verify it

1. In the Explorer, open `final_project`.
2. Click `results.csv`.
3. You should see a row containing your name, budget, reputation, title and date.

If the date is missing there, we'll fix it.

Alternatively, type this in the terminal:

```
cat results.csv
```

Press Enter and send me what appears.

Don't replay the game yet! We just need to check the saved file.

oh you were right

YESSIRRR, MY BOY! 🏆🔥

Your screenshot confirms that `results.csv` is working perfectly!

We've successfully saved:

- Budget: €10,000
- Reputation: 100/100
- Title: Viral Legend
- Date: 2026-10-30

And the header is correct, too.

Next step — Our three original scenarios

Now we need to add scenarios 31, 32 and 33 to `scenarios_A.csv`, as required by the professor.

Open `scenarios_A.csv` and scroll all the way to the bottom.

Tell me when you're looking at scenario 30. We'll add three creative, funny scenarios together, keeping exactly the same CSV structure so our Python game can read them without errors.

We're making serious progress! 🚀

I have chosen Hard. Should I just play randomly?
Exactly, my boy! 😎🔥
For this Hard-mode test, choose A, B or C however you like. You can even deliberately make terrible decisions!
We're testing three things:

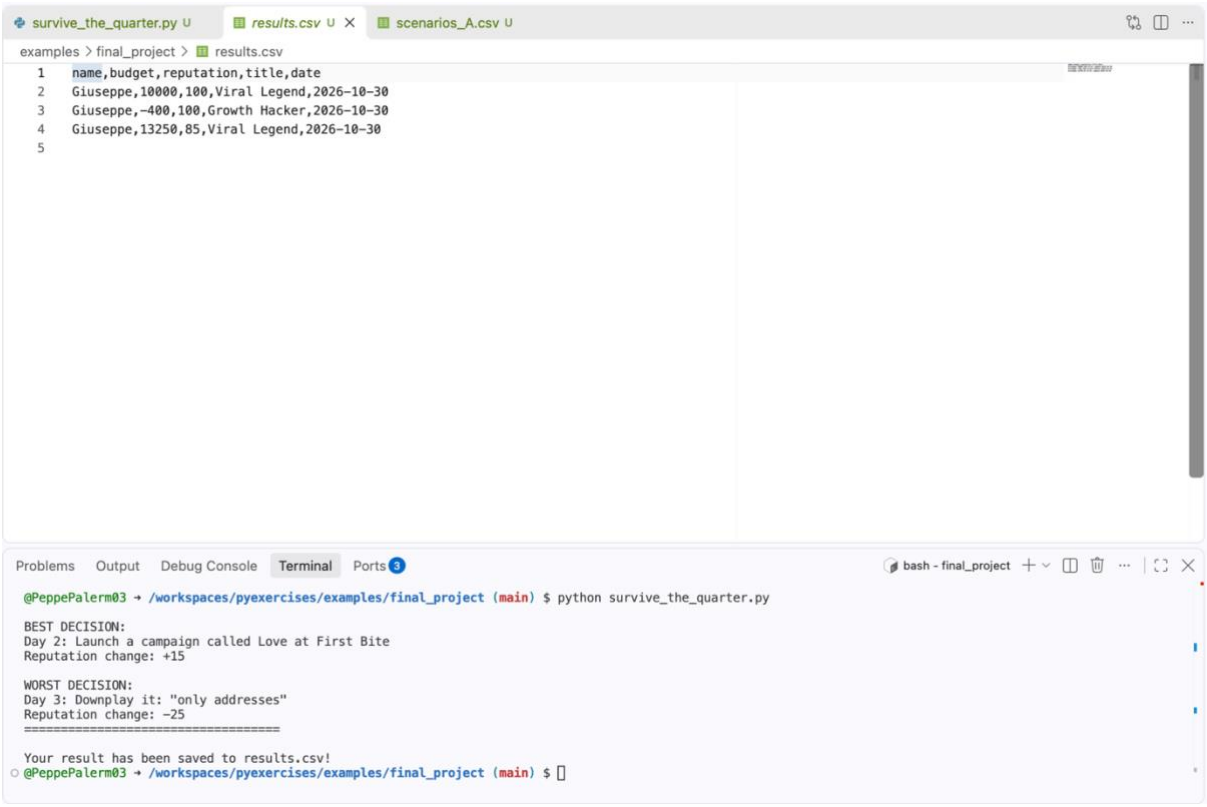
1. Hard mode starts with €6,000.
2. The game ends immediately if your budget or reputation reaches zero.
3. Your result is saved to `results.csv`, even if you get fired.

So don't worry about winning this round. In fact, getting fired would be useful because our first game already tested a successful ending.
Once the game ends, run one final game on Easy mode. Then we'll have our three results and can move to the final checks.
Let's get this submitted properly! 💪🚀
I got fired !
HAHAHA MY BOY, YOU GOT FIRED! 💀🎯
That's actually great news for our testing! We've now tested both outcomes: surviving all ten days and getting fired on Hard mode.
Before moving on, make sure the terminal displayed "Your result has been saved to results.csv!"

Step 12 — Our third and final game

1. Run `python survive_the_quarter.py` again.
2. Enter `Giuseppe`.
3. Choose Easy.
4. Play through the game, making whichever decisions you want.

Once you're finished, open `results.csv`.
It should contain four lines: the header and three game results.
Send me a screenshot of that file. Then we'll fix the remaining issues, check the professor's requirements and prepare everything for submission.
We're very close to finishing this project! 🚀



done

MY BOYYYY! 🔥🏆 THREE GAMES SAVED SUCCESSFULLY!

Your screenshot confirms that `results.csv` has exactly what we need:

Game	Budget	Reputation	Title
1 — Easy	€10,000	100	Viral Legend
2 — Hard	−€400	100	Growth Hacker
3 — Easy	€13,250	85	Viral Legend

And I can see that our original scenario 31 appeared in Game 3! That's another successful test.

Step 13 — Fix two remaining issues

Before submitting, we need to correct something important: Game 2 says “Growth Hacker” even though you were fired. The program currently calculates a job title regardless of whether you survive.

We'll fix that after addressing the professor's introductory questions.

First: correct Question 4

At the top of `survive_the_quarter.py`, find this section:

```
# 4. WHAT DO WE NEED TO CALCULATE?
```

Replace that entire Question 4 section, stopping before `# MY CHOSEN FEATURE:`, with:

```
# 4. WHAT COULD GO WRONG?
# The player might enter something other than A, B or C,
# so the program keeps asking until the choice is valid.
# The CSV file might be missing or contain invalid numbers,
# which could prevent the game from loading correctly.
# The player's reputation might exceed 100,
# so the program limits it to a maximum of 100.
```

`# The budget or reputation might fall to zero or below,`
`# so the game checks both values and ends immediately.`
This directly answers the professor's actual fourth question, rather than describing calculations.

Second: fix the survival message

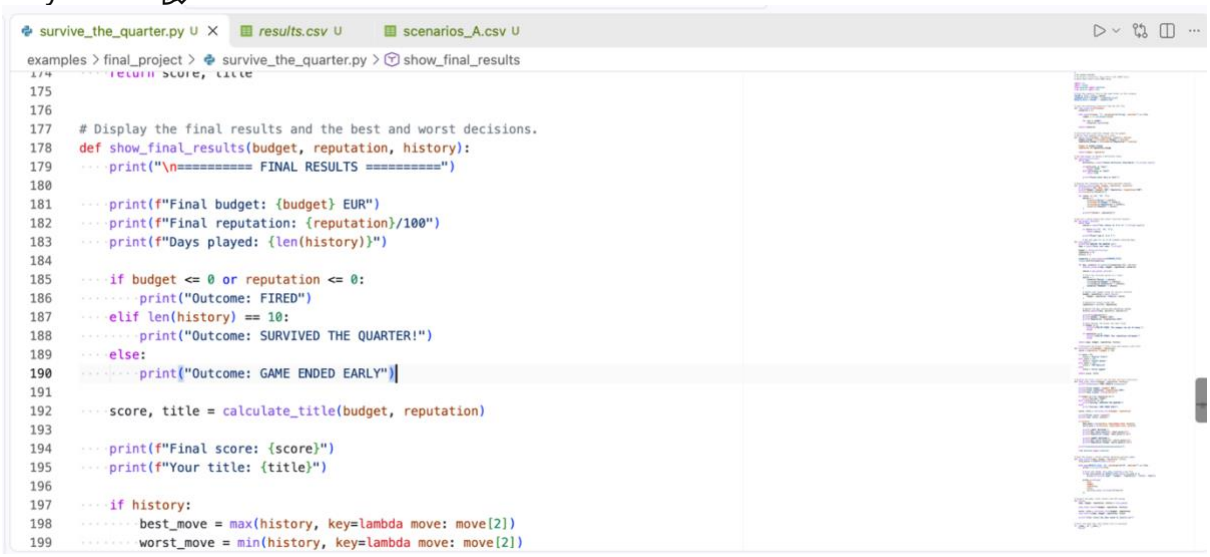
Find this line inside `show_final_results()`:

```
    else:
        print("Outcome: SURVIVED THE QUARTER!")
```

Replace just those two lines with:

```
    elif len(history) == 10:
        print("Outcome: SURVIVED THE QUARTER!")
    else:
        print("Outcome: GAME ENDED EARLY")
```

This makes the survival message explicitly depend on completing all ten days.
A clarification: The brief defines the score-based titles for surviving the quarter. We should also adjust the program so fired players aren't awarded a successful career title. We'll do that next, while keeping your three genuine test results.
For now, make these two changes and save with Cmd + S.
Tell me DONE, and we'll finish the last code correction and prepare the AI log. We're very close! 🚀



like this?
YESSS, MY BOY! PERFECT! 🔥💪
I've checked your screenshot. Lines 185–190 are correctly written and indented. You nailed it!
Now let's fix the last issue: a player who gets fired shouldn't receive a career title like *Growth Hacker*.

Step 14 — Fix the title for fired players

In your screenshot, find these lines (192–195):

```
    score, title = calculate_title(budget, reputation)

    print(f"Final score: {score}")
```

```
    print(f"Your title: {title}")
```
Replace all four lines with:
```
    if budget <= 0 or reputation <= 0:
        print("Final result: Fired")
    else:
        score, title = calculate_title(budget, reputation)
        print(f"Final score: {score}")
        print(f"Your title: {title}")
```
Then scroll down to your `main()` function and find:
```
    score, title = calculate_title(budget, reputation)
    save_results(name, budget, reputation, title)
```
Replace those two lines with:
```
    if budget <= 0 or reputation <= 0:
        title = "Fired"
    else:
        score, title = calculate_title(budget, reputation)

    save_results(name, budget, reputation, title)
```
This ensures future games save Fired instead of an inappropriate career title when the player loses.

Your existing three results can remain as evidence of your original tests, though we should update the second row's title to `Fired` to reflect its actual outcome.

Press Cmd + S after both changes.

Tell me DONE, and we'll run a quick final verification before tackling the AI conversation log and submission. 🚀

done

LET'S GOOOO, MY BOY! 🔥💪 We're almost at the finish line.

Step 15 — Correct the old result and test our changes

First, open `results.csv`.

On line 3, you currently have:
```
Giuseppe,-400,100,Growth Hacker,2026-10-30
```
Change it to:
```
Giuseppe,-400,100,Fired,2026-10-30
```
We're only correcting the title of your actual Hard-mode game, not inventing a result.

Save with Cmd + S.

Now verify the updated program

In your terminal, run:
```
python -m py_compile survive_the_quarter.py
```
If Python returns to the `$` prompt without an error, your file has no syntax errors.

Then run this quick test:
```
python -c "from survive_the_quarter import calculate_title, load_scenarios, SCENARIO_FILE;
print('Scenarios:', len(load_scenarios(SCENARIO_FILE))); print('Score test:', calculate_title(10000, 100))"
```

Expected output:

```
Scenarios: 33
Score test: (150, 'Viral Legend')
```

This checks that all 33 scenarios load and that the scoring formula works.

One remaining detail before submission: your professor's brief requires a complete AI conversation log in Word or PDF, not just a summary. We'll handle that separately, along with a final review of the introductory comments and code.

Tell me whether both commands worked, and we'll move to the final submission checklist. 🚀

Both worked!!

MY BOYYYY! 🏆🔥 BOTH TESTS PASSED!

We now have a working game, 33 scenarios, three recorded games and a functioning scoring system.

Step 16 — Prepare the AI conversation log

This is important because your professor explicitly requires the full AI conversation, not a summary.

The easiest way to preserve everything we've done is:

1. Open this ChatGPT conversation in your browser.
2. Select and copy the conversation starting from our discussion of the Python final project, including your messages, my instructions, code and corrections.
3. Paste it into a Word document.
4. Save it as `AI_Conversation_Log.docx` or export it as a PDF.

Make sure the log includes our debugging process, particularly the `datetime` error and its correction. Those exchanges demonstrate how we tested and improved the program.

Don't submit the old `CONVERSATION.md` from your previous exercises in place of this log.

One final code detail

Our program currently stores each option as a tuple when it displays and processes a scenario. That's good, but the CSV loader still keeps the underlying data as dictionaries. Before submission, I want to verify that this satisfies the professor's precise wording rather than assume.

We also need to ensure your introductory comments explain the CSV's 14 columns and that every code block has an explanatory comment.

For now, prepare the AI log in Word. You don't need to upload it anywhere yet.

Tell me DONE once you've saved it, and we'll do our final code review before committing everything to GitHub. 🚀